# CLOUD-07: CloudWatch Log Ingestion

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 7 of 8 | **Created:** March 2026 | **Last Updated:** 11/10/2026

## Overview

This notebook covers strategies for forwarding cloud provider logs into Dynatrace. While the focus is on AWS CloudWatch log forwarding (the most common pattern), the principles apply to Azure (Event Hub) and GCP (Pub/Sub) as well. You will learn about Amazon Data Firehose (the recommended approach), alternative methods, filtering strategies, OpenPipeline processing, and cost optimization.

---

## Table of Contents


1. Log Ingestion Architecture
2. Amazon Data Firehose (Recommended)
3. Alternative Ingestion Methods
4. Filtering at Source vs Dynatrace
5. OpenPipeline Processing
6. Querying Cloud Logs
7. Cost Optimization
8. Summary and Next Steps


---

# Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | SaaS with Grail enabled |
| **Permissions** | `storage:logs:read` + `storage:buckets:read`, `logs.ingest`; to view or build the §5 pipeline, `settings:objects:read` / `settings:objects:write` (API token: `settings.read` / `settings.write`) — OpenPipeline configuration is Settings 2.0 objects. Changing **routing** needs `settings:objects:write` on `builtin:openpipeline.logs.routing` (routing has no owner-based sharing, so a write grant covers every route in the scope — grant it narrowly) |
| **AWS Integration** | CloudWatch log groups generating data |
| **Prior Knowledge** | CLOUD-01 and CLOUD-02 fundamentals |

> **Sources:** Owner-based access control in OpenPipeline (DT docs) — *"Owner-based control is available for pipelines and ingest sources, and not routing."*, *"This policy gives users write permissions to all routes for a configuration scope and should therefore be used with caution."*

# 1. Log Ingestion Architecture

## AWS CloudWatch Log Forwarding Options

| Method | Mechanism | Latency | Status |
|---|---|---|---|
| **Amazon Data Firehose** | CloudWatch → Firehose → Dynatrace API | 1-5 min (buffered) | **Recommended** |

| Method | Mechanism | Latency | Status |
|---|---|---|---|
| **S3 direct ingestion** | S3 event notification → **Dynatrace-maintained Lambda forwarder** → Dynatrace AWS S3 logs ingest API. Deployed by a single **CloudFormation** stack; records are **linked to their Smartscape entity**. Bypasses CloudWatch and Firehose entirely | Minutes | **Available from SaaS 1.344** — see §3 |
| **Lambda Log Collection** | Direct Lambda function log forwarding | Seconds | Supported (Lambda-specific) |
| **CloudWatch-subscription Lambda forwarder (legacy)** | CloudWatch Subscription Filter → `dynatrace-aws-log-forwarder` Lambda → Dynatrace API | Seconds | **Deprecated** — migrate to Firehose |
| **OpenTelemetry Collector** | FluentBit/FluentD → OTLP → Dynatrace | Seconds | Supported (agent-based) |

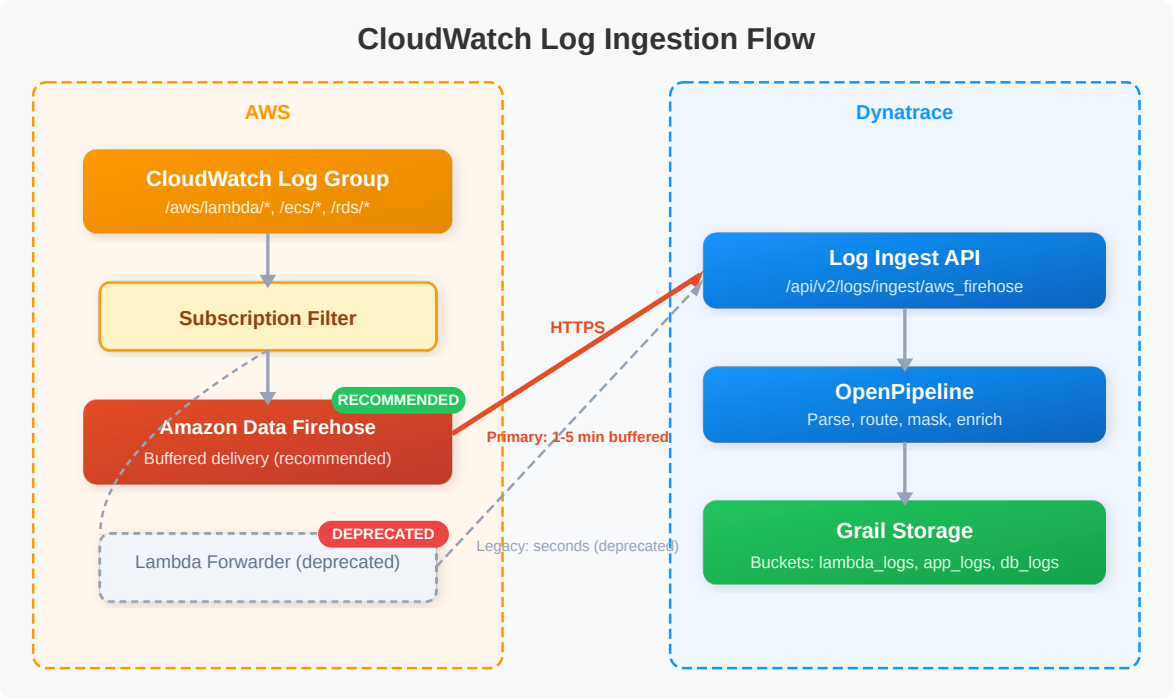
> **Important — what is deprecated, precisely:** the `dynatrace-aws-log-forwarder` Lambda function, on the **CloudWatch Subscription Filter → Lambda → Dynatrace API** path, is **deprecated** and will not receive further updates. For new **CloudWatch** log forwarding, use **Amazon Data Firehose**; existing deployments of that forwarder should plan migration to Firehose.
>
> **This is not a prohibition on Lambda-based log forwarding.** It retires one named component serving one source. The **S3 direct-ingestion** path is a *different*, Dynatrace-maintained Lambda forwarder reading a *different* source, and it is supported — available from **SaaS 1.344** (staged tenant rollout from 07/29/2026). Verify it has reached your tenant before designing around it; see §3 for the pre-1.344 working path.

## Cross-Cloud Log Forwarding

| Cloud Provider | Primary Method | Alternative |
|---|---|---|
| **AWS** | Amazon Data Firehose (CloudWatch logs) | S3 direct ingestion (SaaS 1.344), Lambda Layer log collection |

| Cloud Provider | Primary Method | Alternative |
|---|---|---|
| **Azure** | Azure Event Hub / Diagnostic Settings | Azure Functions forwarder |
| **GCP** | Log Router sink → Pub/Sub → `dynatrace-gcp-monitor` (CLOUD-06 §6) | GCP connection in the Clouds app (GA — verify it has reached your tenant); the Cloud Function deployment is deprecated and unsupported |

### Architecture Overview




## 2. Amazon Data Firehose (Recommended)

Amazon Data Firehose is the recommended method for forwarding CloudWatch logs to Dynatrace. It provides a fully managed, serverless log pipeline with no infrastructure to maintain.

### How It Works

1. **CloudWatch Subscription Filters** are attached to target log groups
2. Logs matching the filter are delivered to an **Amazon Data Firehose** delivery stream
3. Firehose buffers, compresses, and delivers logs to the **Dynatrace Log Ingest API**

4. Logs are processed through **OpenPipeline** and stored in **Grail**

## Key Benefits

| Feature | Description |
|---|---|
| **Fully managed** | No Lambda functions or custom code to maintain |
| **Auto-scaling** | Handles any log volume automatically |
| **Smart retry** | Built-in retry with exponential backoff on delivery failures |
| **Buffering** | Configurable buffer size (1-128 MB) and interval (60-900 seconds) |
| **Compression** | Automatic GZIP compression reduces transfer costs |
| **S3 backup** | Optional backup of all or failed records to S3 |

## Setup via Clouds App

When you onboard an AWS account via the **Clouds app**, CloudWatch log forwarding via Firehose can be configured as part of the guided setup. The CloudFormation template creates:

- Firehose delivery stream configured with Dynatrace endpoint
- IAM roles for Firehose to deliver to Dynatrace
- Secrets Manager entry for the Dynatrace API token
- Optional: CloudWatch subscription filters for selected log groups

## Manual Setup Steps

1. Create a **Firehose delivery stream** with HTTP endpoint destination
2. Set the endpoint URL to the Dynatrace **Firehose** log ingest endpoint: `https://{env-id}.live.dynatrace.com/api/v2/logs/ingest/aws_firehose` — not the generic `/api/v2/logs/ingest` endpoint
3. Configure an **access key** using a Dynatrace API token with the **Ingest logs** permission
4. Set buffer conditions (recommended: 1 MB or 60 seconds)
5. Enable GZIP compression
6. Create **CloudWatch Subscription Filters** on target log groups pointing to the Firehose stream

### Subscription Filter Patterns

CloudWatch subscription filters support pattern matching to pre-filter logs before forwarding:

| Pattern | What It Matches |
|---|---|
| `""` (empty) | All log events |
| `"ERROR"` | Lines containing ERROR |
| `?"ERROR" ?"WARN"` | Lines containing ERROR or WARN |
| `{ $.level = "error" }` | JSON logs where level is error |
| `[ip, id, user, timestamp, request, status_code >= 400]` | Space-delimited logs with status >= 400 |

> **Sources:** Stream logs via Amazon Data Firehose (DT docs) — *"use the full URL:* `https://<your_environment_ID>.live.dynatrace.com/api/v2/logs/ingest/aws_firehose` *"*; the same page gives the **Ingest logs** token permission and the 1 MiB / 60 s buffer hints used in steps 3–4.

## 3. Alternative Ingestion Methods

### S3 Log Ingestion

For logs already stored in S3 (e.g., ALB access logs, CloudTrail, VPC Flow Logs). **The table below describes the Dynatrace-maintained forwarder introduced in SaaS 1.344** — read the rollout note beneath it for what to use until it reaches your tenant:

| Feature | Description |
|---|---|
| **Mechanism** | S3 event notification → **Dynatrace-maintained Lambda forwarder**, deployed in your centralized logging account → Dynatrace **AWS S3 logs ingest API**. The forwarder parses, enriches, and forwards each new object's records |
| **Deployment** | A **single CloudFormation stack blueprint** creates and wires every moving part; version upgrades go through standard CFN stack updates |

| Feature | Description |
|---|---|
| **Entity linking** | When the producing AWS account has an active AWS connection, records from supported sources are **linked to their Dynatrace Smartscape entity** — S3-resident logs land in topology context rather than as orphaned records |
| **Best for** | Historical log import, services that write directly to S3, and bypassing the CloudWatch/Firehose layers to cut cost and operational overhead |
| **Parsing** | Out-of-the-box parsing for common AWS log formats |
| **Multi-region** | Supports cross-region and multi-account setups |

> **Available from SaaS 1.344:** the Dynatrace-maintained S3 forwarder, its CloudFormation blueprint, and Smartscape entity linking arrived with **SaaS 1.344** — **staged tenant rollout** from 07/29/2026. Verify the capability has reached your tenant before designing around it. Until it does, you do not have to build one: **dynatrace-aws-s3-log-forwarder (Dynatrace GitHub)** is Dynatrace-published and actively maintained (verified not archived, 08/27/2026), with out-of-the-box parsing for ELB, CloudFront, CloudTrail, VPC Flow, WAF and MSK, plus cross-region and multi-account rules. ⚠️ **Do not confuse it with** `dynatrace-aws-log-forwarder` — one token shorter, and **archived** (last commit 01/2025). The two names differ by `s3-`, and picking the wrong one lands you on a dead project. The parsing, multi-region and multi-account guidance above applies to **both**.

## Lambda Log Collection

For AWS Lambda function logs specifically, Dynatrace offers direct log collection:

| Feature | Description |
|---|---|
| **Mechanism** | Dynatrace Lambda Layer captures logs directly |
| **Best for** | Lambda-specific monitoring (cost-optimized, lower latency) |
| **Prerequisite** | Dynatrace Lambda Layer already deployed for tracing |
| **Advantage** | No subscription filter or Firehose needed for Lambda logs |

### Legacy: CloudWatch-Subscription Lambda Forwarder (Deprecated)

> **Deprecated:** The `dynatrace-aws-log-forwarder` Lambda function — the **CloudWatch Subscription Filter → Lambda → Dynatrace API** path — is deprecated and will not receive further updates. Migrate existing deployments to Amazon Data Firehose.
>
> **Scope:** this deprecation covers that one component and that one source. It does **not** apply to the **S3 direct-ingestion** forwarder described above, which is a separate, supported, Dynatrace-maintained Lambda function reading S3 rather than CloudWatch.

The legacy Lambda forwarder used CloudWatch Subscription Filters to invoke a custom Lambda function that forwarded logs to Dynatrace. While it provided low-latency delivery, it required maintaining custom Lambda infrastructure.

### Method Comparison

| Method | Latency | Maintenance | Cost | Best For |
|---|---|---|---|---|
| **Firehose** | 1-5 min | None (managed) | Firehose + data transfer | General CloudWatch logs (recommended) |
| **S3 ingestion** | Minutes | Low | S3 storage + transfer | Historical logs, S3-native services |
| **Lambda log collection** | Seconds | None | Included with Lambda Layer | Lambda function logs |
| **CloudWatch-subscription Lambda forwarder** | Seconds | High (deprecated) | Lambda invocations | **Do not use for new CloudWatch forwarding** — this row says nothing about S3 direct ingestion |

## 4. Filtering at Source vs Dynatrace

A critical design decision is **where** to filter logs: at the cloud provider level or within Dynatrace.

## Comparison

| Aspect | Filter at Source (CloudWatch) | Filter in Dynatrace (OpenPipeline) |
|---|---|---|
| **Cost** | Reduces ingestion volume and DPS/DDU cost | Full volume ingested, higher DPS/DDU cost |
| **Flexibility** | Limited pattern matching | Full DQL-based processing |
| **Data loss risk** | Filtered logs are not recoverable | All logs retained (can filter at query time) |
| **Latency** | Immediate (before forwarding) | At ingestion (OpenPipeline processing) |
| **Maintenance** | Subscription filters per log group | Centralized OpenPipeline rules |

## Recommended Hybrid Approach

1. **Source-level (CloudWatch):** Filter out known noise (health checks, debug logs in production)
2. **OpenPipeline:** Enrich, parse, route, and selectively discard or downsample
3. **Query-time:** Use DQL filters for ad-hoc analysis

## Common Source-Level Filters

| Log Group | Filter Pattern | Rationale |
|---|---|---|
| `/aws/lambda/*` | `?"ERROR" ?"WARN" ?"CRITICAL"` | Skip DEBUG/INFO for Lambda |
| `/ecs/service/*` | `""` (all) | Forward all application logs |
| `/aws/rds/*` | `?"error" ?"slow query"` | Focus on errors and slow queries |
| ALB access logs | `[w1, w2, w3, w4, w5, w6, w7, status_code >= 400, ...]` | Only 4xx/5xx responses |

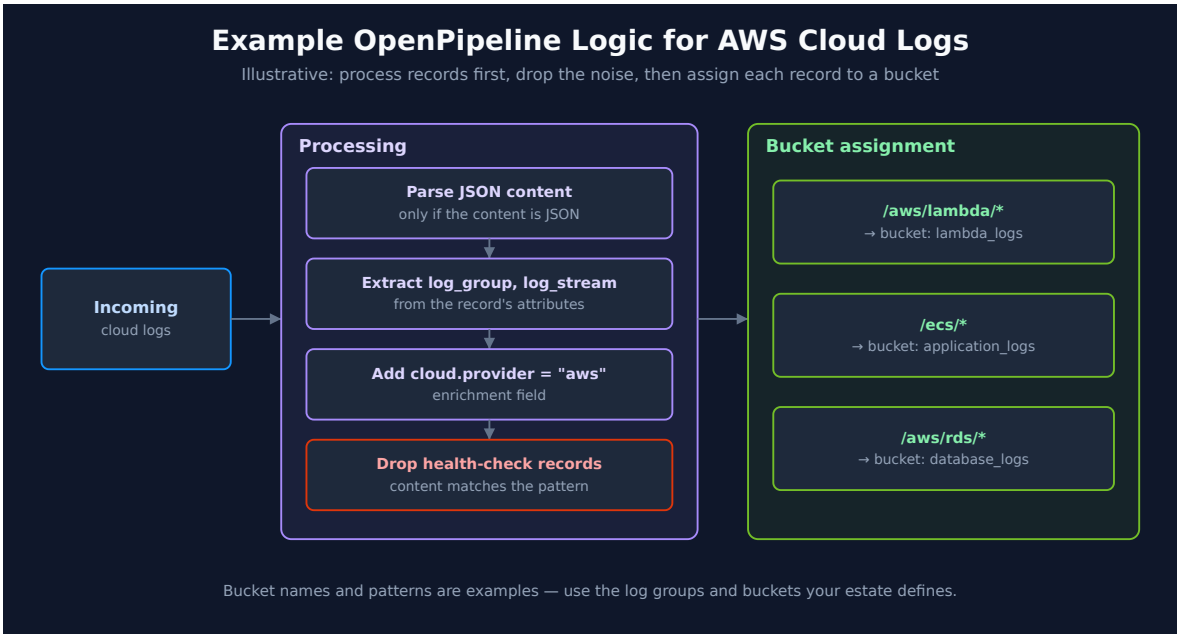
# 5. OpenPipeline Processing

Once cloud logs reach Dynatrace, OpenPipeline processes them before storage in Grail.

## Common OpenPipeline Operations for Cloud Logs

| Operation | Purpose | Example |
|---|---|---|
| **Parse** | Extract structured fields from unstructured logs | Parse JSON, extract IP addresses |
| **Route** | Send logs to specific Grail buckets | Route Lambda logs to `lambda_logs` bucket |
| **Enrich** | Add metadata fields | Add `cloud.provider: aws`, `environment: prod` |
| **Filter/Drop** | Discard noisy log lines | Drop health check entries |
| **Transform** | Modify field values | Normalize severity levels |

## Example Pipeline Logic



# 6. Querying Cloud Logs

## All CloudWatch-Sourced Logs

```
// Recent logs from AWS (last hour)
//
// Corrected 08/12/2026: `contains(log.source, "aws")` returned nothing while
262,723 AWS log
// records sat in the window — CloudWatch-forwarded logs carry the log GROUP in
aws.log_group and
// log.source holds the OneAgent/stream name, which does not contain "aws".
Select on the
// provider field instead.
fetch logs, from:-1h
| filter cloud.provider == "aws"
| fieldsKeep timestamp, content, aws.log_group, log.source, loglevel
| sort timestamp desc
| limit 20
```

## Log Volume by Source

```
// Log volume by source over the last 24 hours
fetch logs, from:-24h
| summarize log_count = count(), by:{log.source}
| sort log_count desc
| limit 15
```

## Error Log Trend Over Time

```
// Error logs per hour over the last 24 hours
fetch logs, from:-24h
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel ==
"ERROR" misses them
| filter status == "ERROR"
| makeTimeseries error_count = count(), interval:1h
```

### Top Error Messages

```
// Most frequent error log messages in the last 6 hours
fetch logs, from:-6h
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel ==
"ERROR" misses them
| filter status == "ERROR"
| summarize error_count = count(), by:{content}
| sort error_count desc
| limit 10
```

### Log Volume by Kubernetes Namespace

```
// Log volume by Kubernetes namespace over the last 6 hours
fetch logs, from:-6h
| filter isNotNull(k8s.namespace.name)
| summarize log_count = count(), by:{k8s.namespace.name, loglevel}
| sort log_count desc
| limit 20
```

## 7. Cost Optimization

Log ingestion is typically the largest cost driver in cloud monitoring. Optimization is critical.

### Cost Impact by Layer

| Layer | Cost Type | Optimization Lever |
|---|---|---|
| **CloudWatch** | Per GB ingested and per GB-month stored (for example $0.50 and $0.03 in us-east-1 for standard logs — check AWS pricing for your region) | Reduce log retention, filter at source |
| **Firehose** | Per GB ingested (check AWS pricing for your region) | Buffer settings, compression |
| **Dynatrace** | DPS/DDU per GB ingested | Source filtering, OpenPipeline drop rules |
| **Grail storage** | DPS/DDU per GB stored x retention | Bucket-level retention policies |

## Optimization Strategies

| Strategy | Where | Complexity |
|---|---|---|
| **Drop debug/trace logs at source** (subscription filter pattern) | AWS | Low |
| **Filter health checks at source** | AWS | Low |
| **Drop remaining noise** with a *Drop record* processor | OpenPipeline | Low |
| **Store low-value logs in a shorter-retention bucket** | OpenPipeline bucket assignment | Medium |
| **Compress Firehose payloads** | AWS | Low |

How much each saves depends entirely on your log mix. In community practice, dropping debug and trace logs at the source is usually the largest single lever — measure your own split first ( `fetch logs, from:-24h | filter cloud.provider == "aws" | summarize n = count(), by:{loglevel}` ). OpenPipeline has no sampling processor for logs; anything you do not want stored is dropped, not sampled.

## Monitoring Your Log Costs

```
// CloudWatch log volume trend by hour over the last 7 days
// Scoped to AWS logs, the subject of this notebook. A 7-day scan of the logs table is billed by
// bytes read, and an unfiltered version of this cell read hundreds of GB on the validation tenant.
// For a trend you look at every day, extract a metric in OpenPipeline instead (FAQ-09).
fetch logs, from:-7d
| filter cloud.provider == "aws"
| makeTimeseries log_count = count(), interval:1h
```

```
// Top 10 CloudWatch log groups by volume over the last 24 hours
fetch logs, from:-24h
| filter cloud.provider == "aws"
| summarize log_count = count(), by:{aws.log_group}
| sort log_count desc
| limit 10
```

# 8. Summary and Next Steps

## Key Takeaways

- **Amazon Data Firehose** is the recommended approach for AWS CloudWatch log ingestion — fully managed, auto-scaling, no custom code
- The legacy **CloudWatch-subscription** Lambda forwarder ( `dynatrace-aws-log-forwarder` ) is **deprecated** — migrate existing CloudWatch deployments to Firehose. This is **not** a blanket rule against Lambda-based forwarding: the **S3 direct-ingestion** forwarder (SaaS 1.344) is a separate, supported component — see §3
- **Filter at the source** (CloudWatch subscription filters) to reduce costs before logs reach Dynatrace
- Use **OpenPipeline** for enrichment, routing, and fine-grained filtering within Dynatrace
- **Monitor log volume** regularly to prevent cost surprises
- Apply similar patterns for **Azure (Event Hub)** and **GCP (Pub/Sub)** log forwarding

## Next Steps

- **CLOUD-08: Multi-Cloud Patterns** — Unified log analysis across providers
- See the **OPLOGS notebook series** for deep dive into OpenPipeline log processing
- See the **OPMIG notebook series** for migrating from classic logs to OpenPipeline

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*